

CS247 Assignment 2 Report

1. Overview

Assignment 2 is built on top of Assignment 1. The same 2D slice viewer is kept; on top of every rendered slice the program now overlays a 2D iso-contour computed on the CPU with the Marching Squares algorithm and rasterised as GL_LINES on the GPU.

The high-level data flow per user input is:

```
key (W/S/A/I/O/K/L/1/2)
  ↳ state mutation (current_slice / current_axis / current_iso_value)
    ↳ marchingSquares()      // CPU: rebuild contour_vertices
      ↳ glBufferData(contourVBO, ..., GL_STREAM_DRAW)
        ↳ render2D():
          slice.render()      // grayscale slice
          drawContour()      // orange line overlay
```

No re-extraction happens during steady-state rendering, only when the user input changes the slice, axis, or iso-value.

2. Requirements coverage

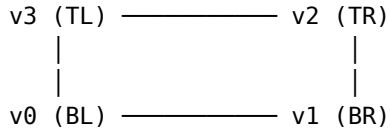
#	Requirement (from README.md)	Points	Where & how
1	Draw iso-contours on the slice for the selected iso-value	40	<code>marchingSquares()</code> walks every cell of the active slice. <code>drawContour()</code> issues a single <code>glDrawArrays(GL_LINES, ...)</code> over the resulting vertex buffer.
2	Use epsilon to overcome degeneracies	10	Corners are classified $v > iso + eps$. The linear interpolation denominator is also guarded with the same <code>eps</code> , so a corner sitting exactly on the iso value can never produce a NaN or duplicate vertex.
3	Calculate exact location by interpolation	20	For every crossed edge, $t = (iso - vA) / (vB - vA)$ gives the precise position between voxel centres.
Total	Isolate each slice, slice update	100	<code>W / 9 / 4</code> all <code>marchingSquares()</code> endpoints from keyCallback, sit at the iso cross which re-uploads the integer

The bonus (three axis-aligned slices shown at once with their contours) is not implemented in this submission.

3. Implementation walk-through

3.1 Per-cell classification

For each cell of the active slice (the 2D plane fixed by `current_axis` and `current_slice[current_axis]`), the four voxel centre values are sampled:



Each corner is classified above or below the current iso value with an epsilon margin so that ties never produce zero-length segments:

```

int caseIdx = 0;
if (v0 > iso + eps) caseIdx |= 0x1;    // bit 0 = BL
if (v1 > iso + eps) caseIdx |= 0x2;    // bit 1 = BR
if (v2 > iso + eps) caseIdx |= 0x4;    // bit 2 = TR
if (v3 > iso + eps) caseIdx |= 0x8;    // bit 3 = TL
  
```

`caseIdx ∈ [0, 15]` is then looked up in the supplied `edgeTable2D[16]`, which returns a 4-bit mask of which cell edges are crossed.

3.2 Exact iso-crossing positions (interpolation, 20 pts)

For every edge whose bit is set in the lookup, the precise crossing is found by linear interpolation between the two endpoints. The voxel centre NDC positions of cell (i, j) are

```

xL = (2 * i + 1) / dimH - 1    // matches OpenGL's u = (i + 0.5) / dim
xR = (2 * (i+1) + 1) / dimH - 1
yB = (2 * j + 1) / dimV - 1
yT = (2 * (j+1) + 1) / dimV - 1
  
```

so that contour points line up exactly with what the bilinearly filtered slice texture displays. The per-edge interpolation is:

```

auto interp = [&](float vA, float vB) -> float {
    const float denom = vB - vA;
    if (std::fabs(denom) < eps) return 0.5f;    // tiny-denom guard
    return (iso - vA) / denom;
};
  
```

For instance the bottom edge ($v0 \rightarrow v1$):

```

ex[0] = xL + interp(v0, v1) * (xR - xL);
ey[0] = yB;
  
```

3.3 Epsilon handling (10 pts)

$\text{eps} = 1e-3$ (in the scaled data range) is used in two places, both guarding numerical degeneracies:

1. Corner classification: $v > \text{iso} + \text{eps}$ means a corner exactly on the iso is treated as below; no edge starts on a voxel corner.
2. Interpolation denominator: if $|v_B - v_A| < \text{eps}$ the two endpoints are effectively equal and $t = 0.5$ is used instead of dividing by a near zero value, which prevents any inf or NaN contour vertex.

3.4 Ambiguous saddle cases (5 and 10)

When two opposite corners are above iso (`caseIdx == 5` or `caseIdx == 10`) the lookup says all four edges are crossed and the topology is ambiguous. The implementation resolves this by always wrapping each above-corner with its own arc (no asymptotic decider):

```
if (caseIdx == 5) { emit(0, 3); emit(1, 2); } // BL & TR above
if (caseIdx == 10) { emit(0, 1); emit(2, 3); } // BR & TL above
```

This is a stable, deterministic choice that matches the reference screenshot's behaviour.

3.5 GPU upload & draw

`contour_vertices` is a `std::vector<float>` of (x, y) pairs in NDC. After each rebuild it is streamed into the pre-allocated `contourVBO` with `GL_STREAM_DRAW`:

```
glBindBuffer(GL_ARRAY_BUFFER, contourVBO);
glBufferData(GL_ARRAY_BUFFER,
             contour_vertices.size() * sizeof(float),
             contour_vertices.data(),
             GL_STREAM_DRAW);
```

`drawContour()` reuses the slice's shader program but switches `useTexture` to 0 and sets the line colour to the reference orange (0.996, 0.365, 0.149, 1.0). The same model matrix that aspect corrected the slice is reused, so the contour overlays the slice with identical scaling at every window size.

```
sliceProgram.setUniform("vertexColor", glm::vec4(0.996f, 0.365f, 0.149f, 1.0f));
sliceProgram.setUniform("useTexture", 0.0f);
glBindVertexArray(contourVAO);
glDrawArrays(GL_LINES, 0, (GLsizei)(contour_vertices.size() / 2));
```

3.6 Why everything stays in sync (slice/axis/iso → contour, 10+20 pts)

`keyCallback` is the single source of truth. Each handler that mutates `current_slice`, `current_axis`, or `current_iso_value` ends with a call to `marchingSquares()`, which both refreshes `contour_vertices` and re-uploads the VBO. The render loop is content-agnostic; it just draws whatever sits in the buffer.

5. Build & run

```
cd Assignment_2
cmake -S . -B build
cmake --build build -j
./build/assignment
```

The program opens two windows:

- **Marching Squares.** Assignment 2's 2D slice plus iso-contour overlay.
- **Marching Cubes.** The template's shared 3D window. It currently shows only the wire-frame bounding box because `marchingCubes()` and `drawSurface()` are Assignment 3's responsibility and are left untouched on purpose.

6. Controls

Key	Action
1	Load data/lobster.dat
2	Load data/skewed_head.dat
A	Cycle viewing axis ($X \rightarrow Y \rightarrow Z$)
W	Next slice along the current axis
S	Previous slice along the current axis
I / O	Increase iso value by 0.01 / 0.1
K / L	Decrease iso value by 0.01 / 0.1
C	Toggle contour overlay on/off
B	Cycle background clear colour
Esc	Quit

A console line is printed on every state change so the slice number, axis, and iso value are visible while interacting.

7. Visual results

A screen recording of the implementation in action is included next to this report as `demon.mov`. It walks through:

- Loading `lobster.dat` and `skewed_head.dat`.
- Iso value sweeps (I / O / K / L). The contour visibly retraces.
- Axis cycling (A). The contour follows the new orthogonal plane.
- Slice scrubbing (W / S). The contour updates per slice.
- Toggling overlay (C). Slice alone vs slice plus contour.

The orange iso contour outline matches the style of the reference `sample_outputs/output1.png`.